

JAVA MODULE-2

A class is a template that defines the form of an object.

A **class** is a **blueprint or template** for creating objects.

An **object** is a **real instance** of a class.

A logical entity that defines properties (variables) and behaviors (methods).

A Java class can contain:

- **Fields (or Instance Variables):** These are variables that represent the state or attributes of an object.
- **Methods:** These are functions that define the behavior or actions an object can perform.
- **Constructors:** Special methods used to initialize objects when they are created.

Defining a class

class is created by using the keyword “class”.

Syntax:

```
accessmodifier class ClassName {  
    // Data members (variables)  
    // Methods (functions)  
}
```

Where class is a keyword and classname is the name of the class.

The access modifier specifies who can access this class.

The class may or may not contain variable declarations or method declarations.

Example:

```
public class Student {  
    int id;  
    String name;  
    void display() {  
        System.out.println(id + " " + name);  
    }  
}
```

Adding Variables (Data Members) to class

Variables inside a class are called **instance variables** or **data members**.

They represent the **state or properties** of an object.

Java Module-2

Ex:

```
class Car {  
    String color;  
    int speed;  
}
```

Adding methods to class

A method is a block of code that performs a specific task.

A method in a class defines a behavior of the object of that class or the behavior of the class itself.

Syntax:

```
Modifiers returnType methodName(parameters) {  
    // method body  
}
```

Example:

```
class Car {  
    String color;  
    int speed;  
    void start() {  
        System.out.println("Car is starting...");  
    }  
    void display() {  
        System.out.println("Color: " + color + ", Speed: " + speed);  
    }  
}
```

In java there are two types of methods:

1. User Defines Methods:

These are methods written by the programmer to perform a specific operation.

They improve code reuse, readability, and structure

Ex: class Example {

```
    void greet() {  
        System.out.println("Hello! Welcome to Java.");  
    }  
    public static void main(String args[]) {
```

Java Module-2

```
Example obj = new Example();  
obj.greet(); // Calling user-defined method  
}  
}
```

2. Predefined (Built-in) Methods:

These are methods provided by Java's standard classes (like math, string, etc)

We can use them directly without defining them ourselves.

Ex:

```
public class BuiltInExample {  
    public static void main(String args[]) {  
        String name = "Chaitra";  
        System.out.println(name.length()); // length() is a predefined method  
        System.out.println(Math.sqrt(25)); // sqrt() is a predefined method  
    }  
}
```

Creating Objects

An object is an instance of a class.

Objects are created using the **new** keyword.

Syntax:

ClassName objectName = new ClassName(arguments) ;

Arguments are optional.

Example:

```
Car c1 = new Car();
```

```
Car c2 = new Car();
```

Accessing class members

Class members (variables and methods) are accessed using the **dot (.) operator**.

Syntax:

To access variables : **object.variable;**

To access Methods: **object.method();**

Example:

```
c1.color = "Red";
```

```
c1.speed = 100;
```

Java Module-2

```
c1.display();
```

```
}
```

Example for accessing variables and methods:

```
class Car {
```

```
String color;    // Data members (variables)
```

```
int speed;
```

```
void display() {    // Method to display car details
```

```
    System.out.println("Color: " + color + ", Speed: " + speed);
```

```
}
```

```
public static void main(String args[]) {    // Main method
```

```
Car c1 = new Car();    // Create object of Car class
```

```
    c1.color = "Red";    // Assign values to object variables using dot operator
```

```
    c1.speed = 100;
```

```
    c1.display();    // Call method to display the values
```

```
}
```

```
}
```

Constructors

A **constructor** is a **special method** in Java used to **initialize objects**.

It is **automatically called** when an object of a class is created.

Constructor name **must be the same** as the **class name**.

No return type (not even **void**).

Syntax:

```
class ClassName {
```

```
    ClassName() {
```

```
        // initialization code
```

```
    }
```

```
}
```

Example:

```
class Student {
```

```
    int id;
```

```
    String name;
```

Java Module-2

```
// Constructor
Student() {
    id = 101;
    name = "Chaitra";
}
void display() {
    System.out.println(id + " " + name);
}
public static void main(String args[]) {
    Student s1 = new Student(); // constructor called automatically
    s1.display();
}
}
```

Characteristics of Constructors

- Automatically invoked when an object is created.
- Cannot be abstract, static, final, or synchronized.
- Used for assigning **initial values** to data members.
- Can be **overloaded** (i.e., multiple constructors in one class).

Types of Constructors

1. Default Constructor
2. No-Argument Constructor
3. Parameterized Constructor

1. Default Constructor

A **no-argument** constructor provided by Java compiler **if no constructor is defined** by user.

It initializes variables with **default values** (0, null, false, etc.)

Ex:

```
class Example {
    int x;
    String y;
    void show() {
        System.out.println(x + " " + y);
    }
}
```

Java Module-2

```
public static void main(String args[]) {  
    Example e = new Example(); // default constructor called  
    e.show();  
}  
}
```

OUTPUT:

0 null

2. No-Argument Constructor

Defined **by the user**, but without parameters.

```
EX: class Car {  
    String color;  
    Car() {  
        color = "Red";  
    }  
    void display() {  
        System.out.println("Color: " + color);  
    }  
    public static void main(String args[]) {  
        Car c1 = new Car();  
        c1.display();  
    }  
}
```

3. Parameterized Constructor

Takes **parameters** to initialize data members with **user-defined values**.

```
class Student {  
    int id;  
    String name;  
    Student(int i, String n) {  
        id = i;  
        name = n;  
    }  
}
```

Java Module-2

```
void display() {  
    System.out.println(id + " " + name);  
}  
  
public static void main(String args[]) {  
    Student s1 = new Student(101, "Asha");  
    Student s2 = new Student(102, "Rahul");  
    s1.display();  
    s2.display();  
}  
}
```

OUTPUT:

```
101 Asha  
102 Rahul
```

Constructor Overloading

Constructor overloading means defining **multiple constructors** in the same class **with different parameter lists** (i.e., different number or types of arguments). It allows you to create objects in **different ways**, providing flexibility in initialization

Example:

```
class Student {  
    int id;  
    String name;  
    // Default constructor  
    Student() {  
        // No initialization here — Java assigns default values automatically  
    }  
    // Parameterized constructor with one argument  
    Student(int i) {  
        id = i;  
        name = "Unknown"; // Assigning a default name  
    }  
    // Parameterized constructor with two arguments  
    Student(int i, String n) {
```

Java Module-2

```
    id = i;
    name = n;
}
void display() {
    System.out.println(id + " " + name);
}
public static void main(String[] args) {
    Student s1 = new Student();    // Calls default constructor
    Student s2 = new Student(101); // Calls 1-parameter constructor
    Student s3 = new Student(102, "Ram");// Calls 2-parameter constructor
    s1.display();
    s2.display();
    s3.display();
}
}
```

OUTPUT:

```
0 null
101 Unknown
102 Ram
```

Method Overloading

Method Overloading is a feature in Java that allows multiple methods with the same name in a class, but different parameter lists.

It helps to **increase the readability** of the program and **reuse method names** for similar operations.

Definition

If a class has two or more methods with the same name but different parameters (number, type, or order), it is called method overloading.

Syntax:

```
class ClassName {
    void methodName() { ... }
    void methodName(int a) { ... }
```


Java Module-2

```
void methodName(int a, int b) { ... }  
}
```

Example:

```
class Addition {  
    // Method 1: No parameters  
    void add() {  
        int a = 10, b = 20;  
        System.out.println("Sum: " + (a + b));  
    }  
    // Method 2: Two integer parameters  
    void add(int a, int b) {  
        System.out.println("Sum: " + (a + b));  
    }  
    // Method 3: Three integer parameters  
    void add(int a, int b, int c) {  
        System.out.println("Sum: " + (a + b + c));  
    }  
    public static void main(String args[]) {  
        Addition obj = new Addition();  
        obj.add();           // calls first method  
        obj.add(10, 30);     // calls second method  
        obj.add(5, 10, 15);  // calls third method  
    }  
}
```

OUTPUT:

Sum: 30

Sum: 40

Sum: 30

Advantages of Method Overloading

1. Improves code readability.
2. Reuses the same method name for similar actions.

Java Module-2

3. Makes the program easier to maintain.
4. Supports compile-time polymorphism.

What are Static Members:

A **static member** (variable or method) belongs to the **class** rather than any specific object. That means it is **shared by all objects** of that class.

It can be:

- ▶ Static variables
- ▶ Static methods

Static Variables:

Static is the keyword.

These are global to a class and belong to the entire set of objects that class creates. Only one memory location is created for each class variable.

They take common value for all the objects.

Example:

```
class Student {  
    int rollNo;  
    String name;  
    static String college = "ABC College"; // Static variable  
    Student(int r, String n) {  
        rollNo = r;  
        name = n;  
    }  
    void display() {  
        System.out.println(rollNo + " " + name + " " + college);  
    }  
    public static void main(String[] args) {  
        Student s1 = new Student(101, "Ravi");  
        Student s2 = new Student(102, "Priya");  
        s1.display();  
    }  
}
```

Java Module-2

```
s2.display();  
}  
}
```

OUTPUT:

101 Ravi ABC College

102 Priya ABC College

Static Methods:

- Static methods are the methods which behave same for all the objects of a class.
- Objects are not needed to call static methods.
- Can be called **without creating an object**.
- Can **access only static variables** directly (not instance variables).

Example:

```
class MathUtils {  
    static int square(int x) {  
        return x * x;  
    }  
    public static void main(String[] args) {  
        System.out.println("Square of 5 = " + MathUtils.square(5));  
    }  
}
```

OUTPUT:

Square of 5 = 25

Nesting methods:

A method can be called by using only its name by another method of the same class. This is called Nesting of methods.

A method calls another method of the same class to perform a task.

Purpose: To divide a large task into smaller, reusable methods.

Java Module-2

Example:

```
class Calculation {
    int add(int a, int b) {
        return a + b;
    }
    void displaySum() {
        int sum = add(10, 20); // Method call inside another method
        System.out.println("Sum = " + sum);
    }
    public static void main(String[] args) {
        Calculation c = new Calculation();
        c.displaySum(); // Starts the chain of method calls
    }
}
```

OUTPUT:

Sum = 30

Here displaySum() method calls add() method. This is nesting methods.

Inheritance (Extending a class)

Inheritance is one of the most important features of Object-Oriented Programming (OOP).

Inheritance is the process of deriving a new class from an existing class. It is always better to use something that already exists rather than writing the same thing from the scratch.

The existing class is known as base or parent or super class.

The new class is known as derived or child or sub class.

The **extend** keyword is used in Java to create a subclass (child class) from a superclass (parent class).

It establishes an inheritance relationship between the two classes.

Syntax:

```
Class derived_classname extends super_class
{
    .....// Data members
    ..... //Member functions
}
```

Java Module-2

Example;

// Parent class

```
class Animal {  
    void eat() {  
        System.out.println("Animals eat food");  
    }  
}
```

// Child class

```
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog barks");  
    }  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.eat(); // Inherited method from Animal  
        d.bark(); // Method of Dog  
    }  
}
```

OUTPUT:

Animals eat food

Dog barks

Types of Inheritance:

There are five types of inheritance:

- Single Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Multiple Inheritance
- Hybrid Inheritance

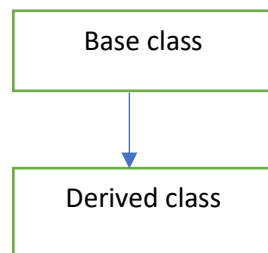
Java Module-2

1. Single Inheritance

If a new class is deriving from an existing class, it can be called as single inheritance.

Single Inheritance means one class inherits from another single class.

That is, there is only one parent class and one child class.



The general form Single inheritance is;

Class baseclass

```
{  
..... // Data members  
..... // Members functions  
}
```

Class derivedclass extends baseclass

```
{  
..... // Data members  
..... // Member functions  
}
```

Example:

// Parent class

```
class Animal {  
    void eat() {  
        System.out.println("Animals can eat");  
    }  
}
```

// Child class

```
class Dog extends Animal {
```

Java Module-2

```
void bark() {  
    System.out.println("Dogs can bark");  
}  
  
public static void main(String[] args) {  
    Dog d = new Dog();  
    d.eat(); // inherited from Animal  
    d.bark(); // defined in Dog  
}  
}
```

OUTPUT:

Animals can eat
Dogs can bark

Here Animal is parent class

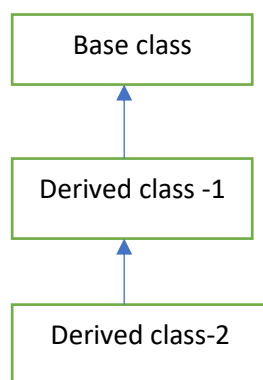
Dog is child class

Dog inherits the eat() method from Animal.

So, Dog can use both eat() and bark() methods.

Multilevel Inheritance

If a class is derived from the class, which is derived from another class, it can be called as multilevel inheritance.



Java Module-2

The general form of multilevel inheritance is;

```
Class baseclass
{
..... // Data members
..... // Member functions
}
Class derivedclass-1 extends baseclass
{
..... // Data members
..... // Member functions
}
Class derivedclass-2 extends derivedclass-1
{
..... // Data members
..... // Member functions
}
```

Example:

```
class Animal {
    void eat() {
        System.out.println("Eating...");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Barking...");
    }
}

class Puppy extends Dog {
    void weep() {
        System.out.println("Weeping...");
    }
}

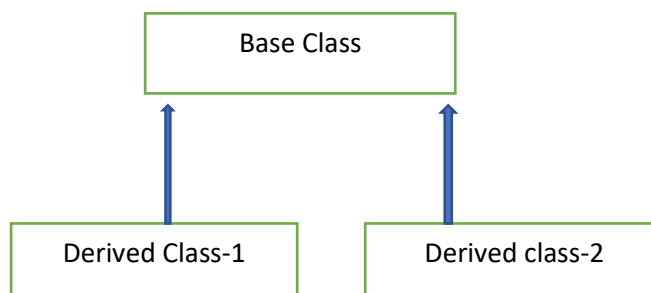
class Test {
    public static void main(String[] args) {
```


Java Module-2

```
Puppy p = new Puppy();  
p.eat();  
p.bark();  
p.weep();  
}  
  
// Puppy inherits Dog, and Dog inherits Animal.  
// So Puppy can access all three methods.
```

3. Hierarchical Inheritance

If two or more classes are derived from a base class, it can be called as hierarchical inheritance.

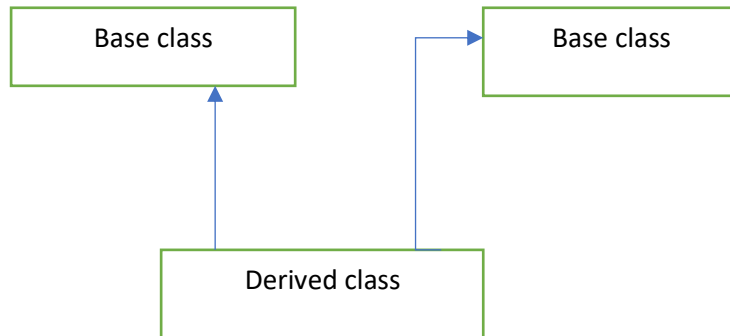


The general form of hierarchical inheritance is,

```
Class baseclass  
{  
..... // Data members  
..... // Member functions  
}  
Class derivedclass-1 extends baseclass  
{  
..... // Data members  
..... // Member functions  
}  
Class derived -2 extends baseclass  
{  
..... // Data members  
..... // Member functions  
}
```

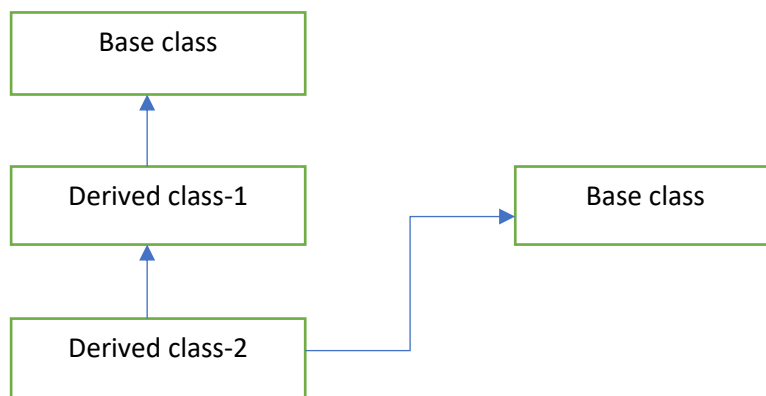
4. Multiple Inheritance

If a class is derived from two or more base classes, it can be called as multiple inheritance. Java does not support multiple inheritance directly. However, it can be implemented using interfaces.



5. Hybrid Inheritance

It is the combination of more than one type of Inheritance. Java does not support hybrid inheritance directly. However, it can be implemented using interfaces.



Java Module-2

Method Overriding:

It occurs when a sub-class(child class) provides its own implementation of a method that is already defined in its superclass (Parent class).

Example:

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes a sound");  
    }  
}  
class Dog extends Animal {  
    // Overriding the parent class method  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}  
public class TestOverride {  
    public static void main(String[] args) {  
        Animal a = new Animal();  
        Dog d = new Dog();  
        a.sound(); // calls Animal's version  
        d.sound(); // calls Dog's version  
    }  
}
```

OUTPUT:

```
Animal makes a sound  
Dog barks
```

The sound() method is defined in both Animal and Dog.

When we create an object of Dog but refer it with Animal a, Java decides at runtime which version to call – this is called runtime polymorphism.

Since the object is of Dog, it calls the overridden method in Dog.

Key Points:

The method in the child class must have the same name, return type and parameters as in the parent class.

There must be inheritance. (using extend).

The child class methods overrides the parent class method.

Differences between Method Overloading and Method Overriding

Feature	Method Overloading	Method Overriding
Definition	Same method name with different parameter lists in the <i>same class</i> .	Same method name, same parameters, but defined in <i>subclass</i> to change parent class behavior.
Concept Type	Compile-time polymorphism (static binding).	Runtime polymorphism (dynamic binding).
Where Implemented?	Within the same class.	Between parent and child classes (inheritance).
Parameters	Must be different (number, type, or order).	Must be exactly the same as the parent class method.
Return Type	Can be same or different.	Must be same or covariant (child type allowed).
Access Specifier	No restriction.	Cannot reduce visibility (e.g., public → private not allowed).
Exception Rules	No special rules.	Cannot throw broader checked exceptions than superclass method.
Inheritance Required?	Not required.	Required (parent–child relationship).
Purpose	To increase readability and flexibility by performing the same action in different ways.	To modify or extend the behavior of inherited methods.
Binding Time	Compile time.	Runtime (decided by JVM at execution).

Java Module-2

Final Variables:

Final variables cannot be altered at later stages in the program once it has been initialized.

These are used to prevent accidental changes to the variables whose values are supposed to be constant throughout.

Ex: PI=3.14 should not be modified in the program at any stage.

Final is the keyword.

Example: final double PI=3.14;

Example:

```
class Test {  
    final int x = 100;        // final instance variable  
    static final double PI = 3.14159; // final static variable  
    void display() {  
        System.out.println("x = " + x);  
        System.out.println("PI = " + PI);  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Test t = new Test();  
        t.display();  
    }  
}
```

OUTPUT:

```
x = 100  
PI = 3.14159
```

Final Methods:

A final method in Java is a method that cannot be overridden by any subclass.

It ensures that the behaviour of the method cannot be changed by child classes.

Example:

```
class Vehicle {  
    final void startEngine() {  
        System.out.println("Engine started!");  
    }  
}
```

Java Module-2

```
    }  
}  
class Car extends Vehicle {  
    // ✗ Cannot override startEngine() because it is final  
}  
public class TestFinalMethod {  
    public static void main(String[] args) {  
        Car myCar = new Car();  
        myCar.startEngine(); // works normally  
    }  
}
```

OUTPUT:

Engine started!

Finalizer() method:

It is called **by the garbage collector** on an object **before the object is destroyed**.

The purpose of calling finalize method is to perform activities related to clean up, resource deallocation etc.,

General form:

[Access specifiers] void finalize()

```
{  
    // finalization code  
}
```

Example:

Class test

```
{  
    public static void main(String args[])  
    {  
        test t=new test();
```

System.gc(); // It is a **request to the JVM** to run the **Garbage Collector (GC)**. Garbage Collector automatically **frees memory** by destroying objects that are no longer reachable.

```
System.out.println("End of main method");
```

```
}  
  
Public void finalize()  
{
```

Java Module-2

```
System.out.println(" Finalize method called");  
}  
}
```

OUTPUT:

End of main method
Finalize method called

Abstract class:

Abstract classes are used to design a common framework comprising a set of methods that have to be defined by all its child classes.

An abstract class is not required to have an abstract method in it. But any class that has an abstract method in it must be declared as an abstract class.

General form is:

```
Abstract class classname  
{  
    //data members  
    //member methods  
}
```

Abstract Methods:

A method that is declared as abstract and does not have implementation is known as abstract method.

The method body will be defined by its subclass.

Abstract method can never be final or static.

General form:

```
Abstract return_type methodname(); // No definition
```

```
// Example program to demonstrate Abstract class and methods
```

```
// Abstract class
```

```
abstract class Animal {
```

```
    // Abstract method
```

```
    abstract void sound();
```

```
}
```

```
// Subclass
```

```
class Dog extends Animal {
```

Java Module-2

```
// Implementing abstract method
void sound() {
    System.out.println("Dog Barks");
}
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.sound(); // Calls implemented abstract method
    }
}
```

OUTPUT:

Dog Barks

Visibility control (Access modifiers or Access Specifiers)

There are four types of access modifiers:

1. Default
2. Public
3. Private
4. Protected

1. Default:

When no access specifier is specified for a class, method or data member. It is said to be having the default access modifier by default.

It is accessible only within the same package.

Ex;

```
class CS
{
    Void display()
    {
        System.out.println("Computer science");
    }
}
```


Java Module-2

2. Public:

The public access modifier has the widest scope among all other access modifiers.

Classes, methods or data members which are declared as public are accessible from everywhere in the program.

There is no restriction on the scope of a public data members.

Ex:

```
class CS
{
public void display()
{
System.out.println("Computer science");
}
}
```

3. Private:

The private access modifier is specified using the keyword private.

The methods or data members declared as private are accessible only within the class in which they are declared.

Any other class of same package will not be able to access these members.

Ex:

```
class CS
{
private void display()
{
System.out.println("Computer science");
}
}
```

4. Protected:

The protected access modifier is specified using the keyword protected.

The methods or data members declared as protected are accessible within same package or subclasses in different package.

Ex:

```
class CS
{
protected void display()
{
System.out.println("Computer science");
}
}
```

Java Module-2

Arrays

An array in Java is a collection of elements of the same data type, stored in contiguous memory locations.

It allows you to store multiple values in a single variable instead of declaring separate variables for each value.

Each element of an array is referred to by specifying the array name followed by one or more subscripts. Each subscript is enclosed in square brackets.

The subscript can begin with number 0.

Array elements are always stored in sequential memory locations.

Advantages

1. Stores multiple elements of the same type efficiently.
2. Provides fast access to elements using index.
3. Easy to traverse using loops.
4. Uses contiguous memory for better performance.
5. Simplifies code for handling fixed-size data.

Disadvantages

1. Fixed size — cannot be changed after creation.
2. Can store only homogeneous (same type) elements.
3. May waste memory if not fully utilized.
4. No built-in methods for insertion or deletion.
5. Insertion and deletion operations are slow.

Types of Arrays in Java

1. **Single-Dimensional Array** – Stores data in a linear form (like a list).

Declaring an Array

Datatype arrayname[]; or datatype[] arrayname;

int[] numbers; // Declaration of an integer array

String[] names; // Declaration of a string array

Creating an Array

After declaring an array, we need to create it in the memory.

Datatype arrayname[]=new datatype[size];

The new keyword is used to create an array object in memory. It allocates space for all elements of the array at runtime. It initializes array elements with default values (e.g., 0 for int, false for boolean, null for objects).

EX; numbers = new int[5]; // Creates an array of size 5

Java Module-2

You can also combine declaration and creation:

```
int[] numbers = new int[5];
```

Initializing an Array

You can assign values individually:

```
numbers[0] = 10;
```

```
numbers[1] = 20;
```

```
numbers[2] = 30;
```

```
numbers[3] = 40;
```

```
numbers[4] = 50;
```

Or use an **array literal**:

```
int[] a= {10, 20, 30, 40, 50};
```

The array reserves 5 storage locations as shown.

a[0]	a[1]	a[2]	a[3]	a[4]
10	20	30	40	50

Accessing Array Elements

Array indices start from **0**.

```
System.out.println(numbers[2]); // Output: 30
```

Array Length

In java, all arrays store the allocated size in a variable named length. We can obtain the length of the array using length.

```
int a[]={10,20,30,40,50}
```

```
int n=a.length; // the value of n is 5
```

Example program:

```
class OneDArrayExample {  
    public static void main(String[] args) {  
        // Declaration and creation of array  
        int[] numbers = new int[5];  
        // Initialization of array elements  
        numbers[0] = 10;  
        numbers[1] = 20;
```

Java Module-2

```
numbers[2] = 30;
numbers[3] = 40;
numbers[4] = 50;

// Displaying array elements
System.out.println("Elements of the array are:");
for (int i = 0; i < numbers.length; i++) {
    System.out.println(numbers[i]);
}
}
```

OUTPUT:

Elements of the array are:

10
20
30
40
50

2. Program to print names

```
class StringArrayForLoop {
    public static void main(String[] args) {
        // Declaration and initialization of string array
        String[] names = {"Alice", "Bob", "Charlie", "David"};
        System.out.println("Names in the array:");
        // Using normal for loop
        for (int i = 0; i < names.length; i++) {
            System.out.println(names[i]);
        }
    }
}
```

OUTPUT:

Names in the array:

Alice

Java Module-2

Bob

Charlie

David

```
import java.util.Scanner;
```

//Program to read string array values from keyboard

```
class StringArrayInput {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        // Ask user for number of names  
        System.out.print("Enter the number of names: ");  
        int n = sc.nextInt();  
        sc.nextLine(); // consume the leftover newline  
        // Declare string array  
        String[] names = new String[n];  
        // Read names from user  
        System.out.println("Enter the names:");  
        for (int i = 0; i < n; i++) {  
            System.out.print("Name " + (i + 1) + ": ");  
            names[i] = sc.nextLine();  
        }  
        // Display the entered names  
        System.out.println("\nNames you entered are:");  
        for (int i = 0; i < n; i++) {  
            System.out.println(names[i]);  
        }  
        sc.close();  
    }  
}
```

NOTE:

The nextInt() does not consume the newline character (**\n**) that occurs when you press Enter after typing the number. Always use sc.nextLine() after sc.nextInt() to clear the newline.

Java Module-2

Two Dimensional array

Arrays which have elements with two subscripts are known as 2-D array.

It consists of rows and columns.

Declaring the array

```
Datatype arrayname[][];
```

Or

```
Datatype [][] arrayname;
```

Ex:

```
int a[][];
```

```
int [][] total;
```

```
float average[][];
```

Creating memory locations:

```
Datatype arrayname[][];
```

```
Arrayname =new datatype[size1][size2];
```

Or

```
Datatype arrayname=new datatype[size1][size2];
```

Ex:

```
int a[][]=new int[2][2];
```

a	[0]	[1]
[0]	a[0][0]	a[0][1]
[1]	a[1][0]	a[1][1]

Initialization of values

```
Datatype arrayname[][]={list of values};
```

Ex:

```
int a[][]={{10,20},{30,40}};
```

Java Module-2

//Program to display matrix

```
class TwoDArrayExample {  
    public static void main(String[] args) {  
        int[][] matrix = {  
            {10, 20, 30},  
            {40, 50, 60},  
            {70, 80, 90}  
        };  
        System.out.println("Matrix elements are:");  
        for (int i = 0; i < 3; i++) {          // rows  
            for (int j = 0; j < 3; j++) {      // columns  
                System.out.print(matrix[i][j] + " ");  
            }  
            System.out.println(); // move to next line after each row  
        }  
    }  
}
```

OUTPUT:

Matrix elements are:

10 20 30

40 50 60

70 80 90

// Program to find product of two matrices

```
import java.util.Scanner;  
public class MatrixMultiplication {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
  
        // Input dimensions of first matrix  
        System.out.print("Enter rows of first matrix: ");  
        int r1 = sc.nextInt();  
        System.out.print("Enter columns of first matrix: ");  
        int c1 = sc.nextInt();  
  
        // Input dimensions of second matrix  
        System.out.print("Enter rows of second matrix: ");
```

Java Module-2

```
int r2 = sc.nextInt();
System.out.print("Enter columns of second matrix: ");
int c2 = sc.nextInt();

// Check if multiplication is possible
if (c1 != r2) {
    System.out.println("Matrix multiplication not possible. Columns of first must equal rows of second.");
    return;
}
int[][] A = new int[r1][c1];
int[][] B = new int[r2][c2];
int[][] product = new int[r1][c2];
// Input first matrix
System.out.println("Enter elements of first matrix:");
for (int i = 0; i < r1; i++) {
    for (int j = 0; j < c1; j++) {
        A[i][j] = sc.nextInt();
    }
}
// Input second matrix
System.out.println("Enter elements of second matrix:");
for (int i = 0; i < r2; i++) {
    for (int j = 0; j < c2; j++) {
        B[i][j] = sc.nextInt();
    }
}
// Matrix multiplication
for (int i = 0; i < r1; i++) {
    for (int j = 0; j < c2; j++) {
        for (int k = 0; k < c1; k++) {
            product[i][j] += A[i][k] * B[k][j];
        }
    }
}
// Print product matrix
System.out.println("Product of the matrices:");
for (int i = 0; i < r1; i++) {
    for (int j = 0; j < c2; j++) {
        System.out.print(product[i][j] + " ");
    }
    System.out.println();
}

sc.close();
}
```


Java Module-2

Strings

A **String** is a **sequence of characters** enclosed within double quotes (" ").

Ex: String name = "Alice";

Declaration:

String stringname;

Stringname="string";

Or

String stringname="string";

Ex:

String college="GCC";

- Using new Keyword

String s2 = new String("Hello");

String methods:

1. toLowerCase() – Converts all the characters of the string in lower case.

Ex:

String str="COMPUTER";

Str.toLowerCase(); // computer

2. toUpperCase() - Converts all the characters of the string to Uppercase.

Ex:

String str="computer";

Str.toUpperCase(); //COMPUTER

3. length() – Returns the length of the string.

Ex:

String str="computer";

int a=str.length(); //8

4. concat() – Concatenates two strings

Ex: String s1="computer";

String s2="science";

S3=str1.concat(str2); //computerscience

5. charAt() – Returns a character of the string at nth position, as it starts counting from 0th position.

Ex: str="computer"

Str.charAt(5); //t

Java Module-2

6. substring() – Returns the substring starting from the mth character of the string.

```
Str="computer";
```

```
Str.substring(3); // puter
```

7. equals() – Checks whether two strings are equal or not

```
Ex: String s1="computer";
```

```
String s2="computer";
```

```
S1.equals(s2); //true
```

//Write a program to implement all string operations.

```
import java.util.*;
```

```
public class StringOperations {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        // Input two strings
```

```
        System.out.print("Enter first string: ");
```

```
        String str1 = sc.nextLine();
```

```
        System.out.print("Enter second string: ");
```

```
        String str2 = sc.nextLine();
```

```
        // 1. Length of strings
```

```
        System.out.println("\nLength of first string: " + str1.length());
```

```
        System.out.println("Length of second string: " + str2.length());
```

```
        // 2. Concatenation
```

```
        String concat = str1 + str2;
```

```
        System.out.println("\nConcatenation of both strings: " + concat);
```

```
        // 3. Character access
```

```
        if (str1.length() > 0)
```

```
            System.out.println("First character of first string: " + str1.charAt(0));
```

Java Module-2

```
if (str2.length() > 0)

    System.out.println("Last character of second string: " + str2.charAt(str2.length() - 1));

// 4. Substring

if (str1.length() >= 3)

    System.out.println("\nSubstring of first string (0-3): " + str1.substring(0, 3));

// 5. String comparison

System.out.println("\nString comparison using equals(): " + str1.equals(str2));

// 6. Case conversion

System.out.println("\nUppercase of first string: " + str1.toUpperCase());

System.out.println("Lowercase of second string: " + str2.toLowerCase());

sc.close();

}

}
```

OUTPUT:

```
C:\java>java StringOperations
Enter first string: chaitra
Enter second string: GOWDA

Length of first string: 7
Length of second string: 5

Concatenation of both strings: chaitraGOWDA
First character of first string: c
Last character of second string: A

Substring of first string (0-3): cha

String comparison using equals(): false

Uppercase of first string: CHAITRA
Lowercase of second string: gowda
```

Vectors

Vector is a dynamic array that can grow or shrink in size as elements are added or removed. It is part of the java.util package and extends the AbstractList class.

- Maintains insertion order and allows duplicate and null values.
- Dynamically grows its size when capacity is exceeded.

General Syntax for Declaring a Vector

```
Vector<datatype> v = new Vector<>();
```

Constructors in Vectors

- **Vector():** A default vector of capacity 10 gets created while calling this constructor.
- **Vector(int size):** A vector is created with the given size as its capacity.
- **Vector(int size, int increment):** A vector is created with the given size as its initial capacity, and whenever the capacity needs to be increased, it is increased by the given increment count.
- **Vector(Collection c):** A Java vector is constructed from the given collection with the same order of elements as in the collection.

Vector Methods

Method	Description	Example
add(element)	Adds an element to the Vector	v.add("Java");
add(index, element)	Inserts element at a specific index	v.add(1, "C++");
get(index)	Returns the element at given index	v.get(0);
set(index, element)	Replaces element at the specified index	v.set(1, "HTML");
remove(index)	Removes element at the given index	v.remove(2);
removeElement(element)	Removes first occurrence of the element	v.removeElement("Java");
removeAllElements()	Removes all elements (clears vector)	v.removeAllElements();
clear()	Clears all elements (same as removeAllElements)	v.clear();
size()	Returns number of elements in the Vector	int n = v.size();
capacity() Returns current capacity of the Vector	Returns current capacity of the Vector	int c = v.capacity();

Java Module-2

isEmpty()	Checks if Vector is empty	v.isEmpty();
firstElement()	Returns the first element	v.firstElement();
lastElement()	Returns the last element	v.lastElement();
elementAt(index)	Returns element at index	v.elementAt(1);
insertElementAt(el, index)	Inserts element at given index	v.insertElementAt("DBMS", 2);
removeElementAt(index)	Removes element at given index	v.removeElementAt(1);

//Example program to demonstrate Vectors

```
import java.util.*;

class VectorMethodsDemo {
    public static void main(String[] args) {
        Vector<String> v = new Vector<>();
        v.add("Java");
        v.add("Python");
        v.add("C++");
        v.add(1, "HTML");
        System.out.println("Vector: " + v);
        System.out.println("Size: " + v.size());
        System.out.println("Capacity: " + v.capacity());
        System.out.println("First Element: " + v.firstElement());
        System.out.println("Last Element: " + v.lastElement());
        v.remove("HTML");
        System.out.println("After Removal: " + v);
        if (v.contains("Java")) {
            System.out.println("Java found!");
        }
    }
}
```

Java Module-2

Wrapper Classes

Wrapper classes provide a way to use primitive data types (int, boolean, etc..) as objects.

Wrapper Classes are special classes that convert primitive data types (int, float, boolean, etc.) into objects.

Java is an Object-Oriented Programming (OOP) language, and many features in Java work only with objects.

Primitive types, however, are not objects, so wrapper classes act as a bridge between primitive data and object-based features.

Features:

It converts numeric strings into numeric values.

The way to store primitive data in an object.

All wrapper class have `typeValue()` method. This method returns the value of the object as its primitive type.

Method Name	Purpose / Description	Example
<code>valueOf()</code>	Converts a primitive or String into a wrapper object.	<code>Integer i = Integer.valueOf("123");</code>
<code>parseXxx()</code> (<code>parseInt</code> , <code>parseDouble</code> , etc.)	Converts a string into a primitive type.	<code>int n = Integer.parseInt("100");</code>
<code>xxxValue()</code> (<code>intValue</code> , <code>floatValue</code> , etc.)	Converts a wrapper object back to primitive.	<code>int x = i.intValue();</code>
<code>compareTo()</code>	Compares two wrapper objects. Returns +, -, or 0.	<code>a.compareTo(b);</code>
<code>compare()</code>	Compares two primitive values using wrapper class.	<code>Integer.compare(10, 5);</code>
<code>equals()</code>	Checks if two wrapper objects have equal values.	<code>a.equals(b);</code>
<code>toString()</code>	Converts wrapper object to String.	<code>String s = i.toString();</code>
<code>MAX_VALUE</code>	Returns maximum value of the data type.	<code>Integer.MAX_VALUE</code>
<code>MIN_VALUE</code>	Returns minimum value of the data type.	<code>Integer.MIN_VALUE</code>
<code>isDigit()</code>	Checks if character is a digit.	<code>Character.isDigit('5');</code>
<code>isLetter()</code>	Checks if character is a letter.	<code>Character.isLetter('A');</code>
<code>isUpperCase()</code>	Checks if character is uppercase.	<code>Character.isUpperCase('A');</code>
<code>isLowerCase()</code>	Checks if character is lowercase.	<code>Character.isLowerCase('a');</code>
<code>isWhitespace()</code>	Checks if character is space/tab/newline.	<code>Character.isWhitespace(' ');</code>

Java Module-2

// Program to demonstrate Wrapper class

```
class WrapperMethodDemo {  
    public static void main(String[] args) {  
        // valueOf()  
        Integer a = Integer.valueOf("100");  
        Double b = Double.valueOf("45.67");  
        System.out.println("valueOf() method:");  
        System.out.println("Integer value: " + a);  
        System.out.println("Double value: " + b);  
        // parseXxx()  
        int p = Integer.parseInt("200");  
        double q = Double.parseDouble("55.55");  
        System.out.println("\nparseXxx() method:");  
        System.out.println("Parsed int: " + p);  
        System.out.println("Parsed double: " + q);  
        // xxxValue()  
        Integer obj = 150;  
        System.out.println("\nxxxValue() method:");  
        System.out.println("intValue(): " + obj.intValue());  
        System.out.println("floatValue(): " + obj.floatValue());  
        // compareTo()  
        Integer x = 30, y = 40;  
        System.out.println("\ncompareTo() method:");  
        System.out.println("x.compareTo(y): " + x.compareTo(y)); // -1  
        // equals()  
        Integer m = 50, n = 50;  
        System.out.println("\nequals() method:");  
        System.out.println("m.equals(n): " + m.equals(n)); // true  
        // toString()  
        Integer t = 999;  
        System.out.println("\ntoString() method:");  
        System.out.println("String value: " + t.toString());  
    }  
}
```

Java Module-2

```
// MAX_VALUE & MIN_VALUE
System.out.println("\nMAX_VALUE and MIN_VALUE:");
System.out.println("Integer.MAX_VALUE: " + Integer.MAX_VALUE);
System.out.println("Integer.MIN_VALUE: " + Integer.MIN_VALUE);
// Character class methods
System.out.println("\nCharacter class methods:");
System.out.println("isDigit('5'): " + Character.isDigit('5'));
System.out.println("isLetter('A'): " + Character.isLetter('A'));
System.out.println("isUpperCase('A'): " + Character.isUpperCase('A'));
System.out.println("isLowerCase('a'): " + Character.isLowerCase('a'));
System.out.println("isWhitespace(' '): " + Character.isWhitespace(' '));
}
}
```

OUTPUT:

valueOf() method:

Integer value: 100

Double value: 45.67

parseXxx() method:

Parsed int: 200

Parsed double: 55.55

xxxValue() method:

intValue(): 150

floatValue(): 150.0

compareTo() method:

x.compareTo(y): -1

equals() method:

m.equals(n): true

toString() method:

String value: 999

MAX_VALUE and MIN_VALUE:

Integer.MAX_VALUE: 2147483647

Integer.MIN_VALUE: -2147483648

Java Module-2

Character class methods:

isDigit('5'): true

isLetter('A'): true

isUpperCase('A'): true

isLowerCase('a'): true

isWhitespace(' '): true